

LlamaIndex Integration

Compress RAG context and chat memory inside LlamaIndex with a single drop-in component. Smaller prompts, lower LLM spend, faster responses, no changes to your retrievers or vector store.

1 Install

```
pip install 'compresr[llamaindex]'
export COMPRESR_API_KEY=cmp...
```

Requires `llama-index-core>=0.11`. The memory block additionally requires the `Memory API` (`llama-index-core>=0.12`).

2 Drop-in node postprocessor

Add `CompresrNodePostprocessor` to `as_query_engine()` and every retrieved node is rewritten with query-aware compression before the response synthesizer sees it. The query is read automatically from the active query engine.

```
import os
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from compresr.integrations.llamaindex import CompresrNodePostprocessor

documents = SimpleDirectoryReader("docs/").load_data()
index = VectorStoreIndex.from_documents(documents)

compresr_pp = CompresrNodePostprocessor(
    api_key=os.environ["COMPRESR_API_KEY"],
    compression_model="latte_v1",      # query-specific model
    target_compression_ratio=0.5,      # remove ~50% of tokens per node
    min_tokens=200,                   # skip short nodes
)

query_engine = index.as_query_engine(
    similarity_top_k=10,
    node_postprocessors=[compresr_pp],
)

response = query_engine.query(
    "What did the customer say about enterprise pricing?"
)
```

Supports adaptive output budgets via `target_token`. Fail-open by default: if the Compresr API is unavailable, your pipeline continues with the original nodes.

3 Memory block

Plug `CompresrMemoryBlock` into the LlamaIndex 2026 `Memory API` to shrink chat history in place — no extra LLM summarization call, no dropped messages.

```
import os
from llama_index.core.memory import Memory
from compresr.integrations.llamaindex import CompresrMemoryBlock

memory = Memory.from_defaults(
    session_id="user-42",
    token_limit=8_000,
    memory_blocks=[
        CompresrMemoryBlock(
            api_key=os.environ["COMPRESR_API_KEY"],
            target_token=2_000,      # output budget in tokens
            min_tokens=200,
        )
    ],
)
```

4 Tool wrapper

Wrap any `FunctionTool` so its return value is compressed before reaching the agent. Metadata (name, description, schema) is preserved.

```
from llama_index.core.tools import FunctionTool
from compresr.integrations.llamaindex import wrap_tool_with_compresr

def search_crm(query: str) -> str:
    """Search the CRM for a customer."""
    return _do_search(query)

tool = FunctionTool.from_defaults(fn=search_crm)
wrapped = wrap_tool_with_compresr(
    tool,
    api_key=os.environ["COMPRESR_API_KEY"],
    target_compression_ratio=0.5,
    query_arg="query",      # use the tool's `query=` arg as compression query
)

from llama_index.core.agent.workflow import FunctionAgent
agent = FunctionAgent(tools=[wrapped], llm=llm)
```

5 Key parameters

Parameter	Default	Description
<code>api_key</code>	—	Your Compresr API key. Or pass a pre-built <code>client</code> .
<code>compression_model</code>	<code>"latte_v1"</code>	<code>latte_v1</code> (default, query-specific) or <code>latte_v2</code> (faster variant).
<code>target_compression_ratio</code>	0.5	[0, 1] = fraction of tokens to remove (0.5 = remove ~50%). > 1 = Nx factor (4 = ~4x smaller).
<code>target_token</code>	<code>None</code>	Absolute output token budget per node or buffer. When set, overrides <code>target_compression_ratio</code> .
<code>min_tokens</code>	200	Skip compression when input is below this threshold. Token count is estimated as $\sim \text{chars} / 4$ — close enough to filter small payloads without an extra tokenizer call.
<code>coarse</code>	<code>None</code>	<code>True</code> = paragraph-level (faster). <code>False</code> = token-level (finer). <code>None</code> leaves the choice to the server's per-model default.
<code>query</code> / <code>query_arg</code> / <code>query_extractor</code>	<code>None</code>	Override or extract the compression query (tool wrapper). Default lifts <code>query</code> , <code>question</code> , or similar kwargs automatically. <code>query_extractor</code> is a callable <code>(kwargs: dict) -> str None</code> — return <code>None</code> to fall back to the auto-default.
<code>on_error</code>	<code>"passthrough"</code>	<code>"passthrough"</code> = log and return original content on failure. <code>"raise"</code> = propagate.